Ploting

Jupyter notebooks

This is a space where text and code come together to form a coherent document. Throughout the course, we will make use of Jupyter notebooks for illustration purposes and for experimentation.

Plotting

Plotting is very useful to visualize data and can be easily done via the `matplotlib` library in Python. To perform some plotting, we need (at the very least) 2 lists `X` and `Y` that represent the x-coordinate and y-coordinate values, respectively. We also need to write one import statement, and we are all set.

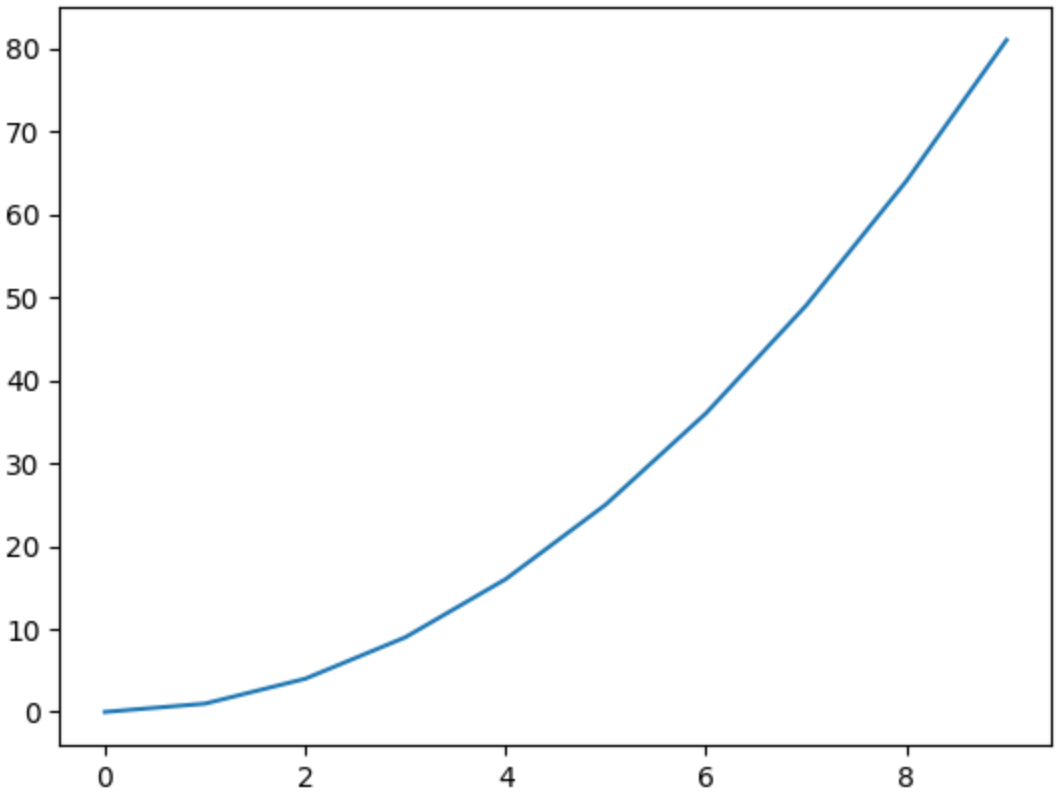
Let's plot the function `y = x^2` for `x = 0, 1, 2, ... , 9`. Using list-comprehension, we create the required lists:

In [1]:
```
import matplotlib.pyplot as plt

X = [i for i in range(0, 10)]
Y = [x**2 for x in X]

plt.plot(X, Y)
```

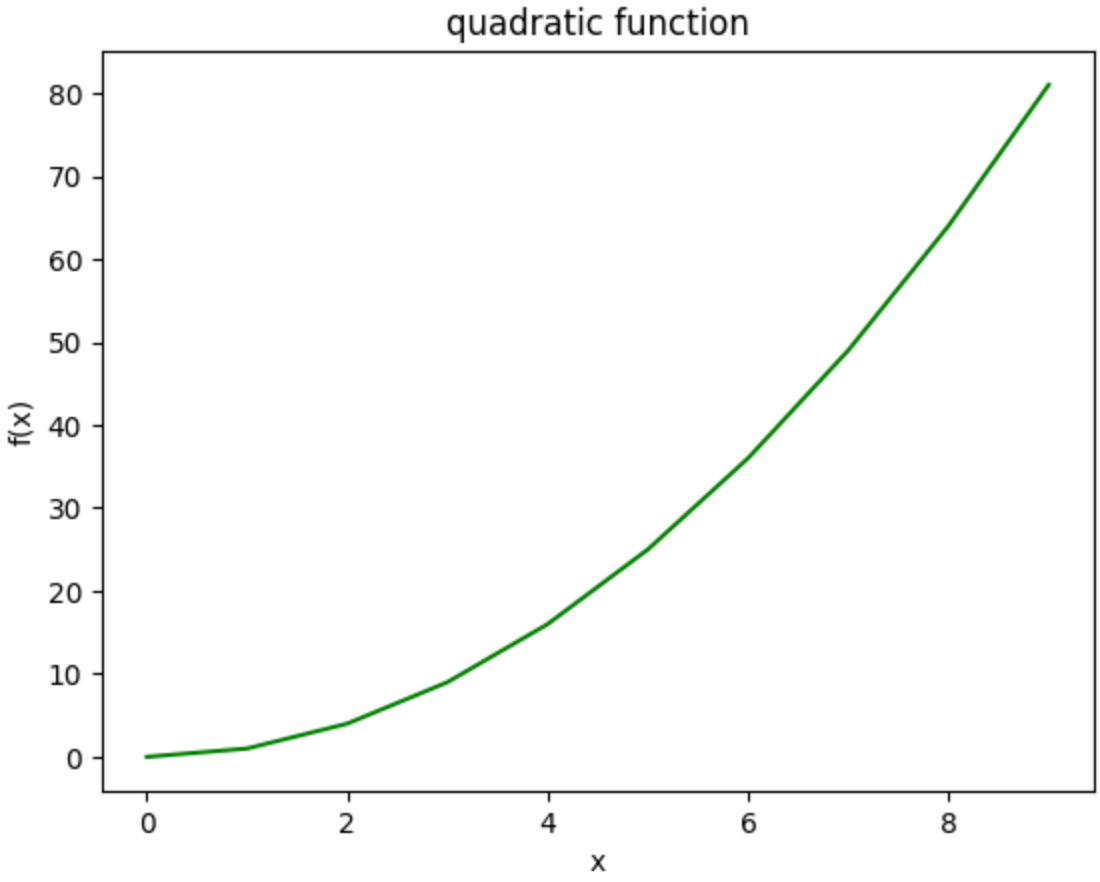
Out[1]:
```
[<matplotlib.lines.Line2D at 0x2a28e2dbee0>]
```



To customize our plot, we can invoke additional commands that take strings as inputs to display the x-labels, y-labels, and give our figure a title.

In [2]:
```
plt.plot(X, Y, color='g')
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("quadratic function")
```

Out[2]:
```
Text(0.5, 1.0, 'quadratic function')
```

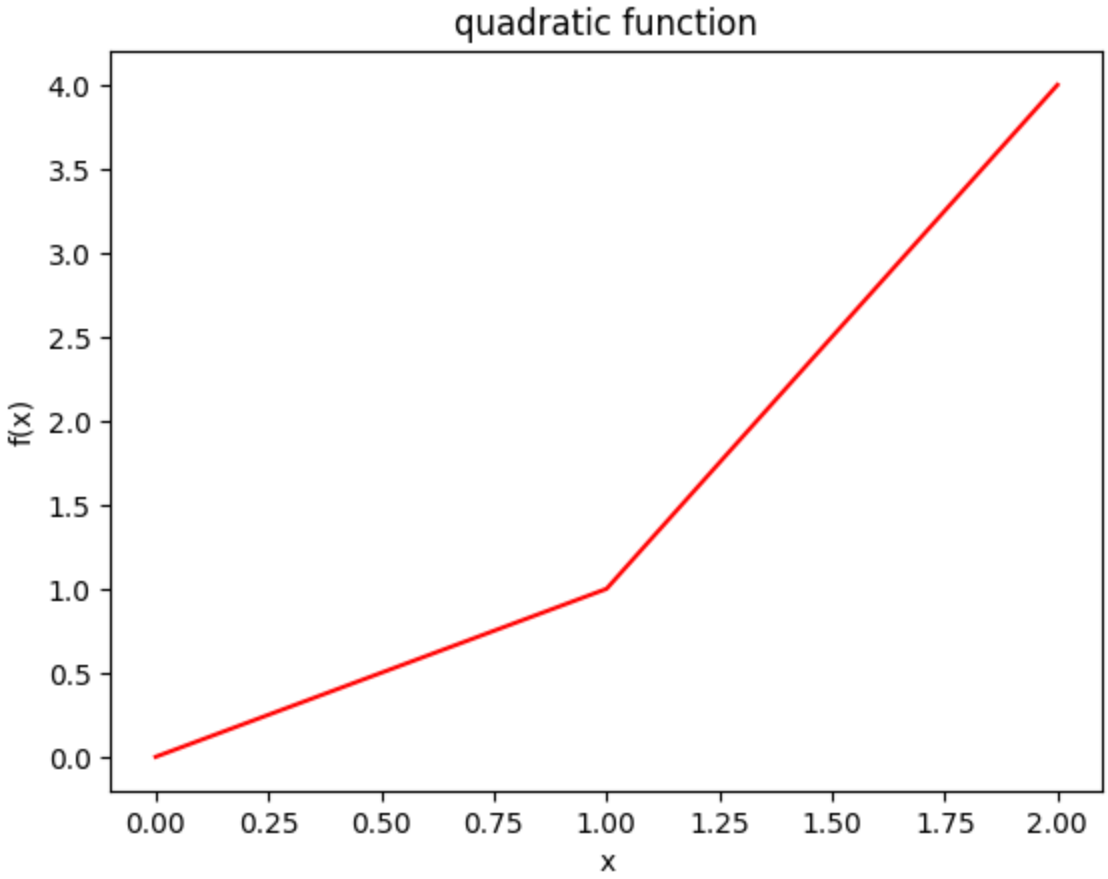


Defining the x_coordinate range

What if we need to plot `y = x^2` in a small interval? Example, for `0 <= x <= 2`. Using our usual method, we only have 3 data points:

In [3]:
```
X = [i for i in range(0, 3)]
Y = [x**2 for x in X]
plt.plot(X, Y, 'red')
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("quadratic function")
```

Out[3]:
```
Text(0.5, 1.0, 'quadratic function')
```

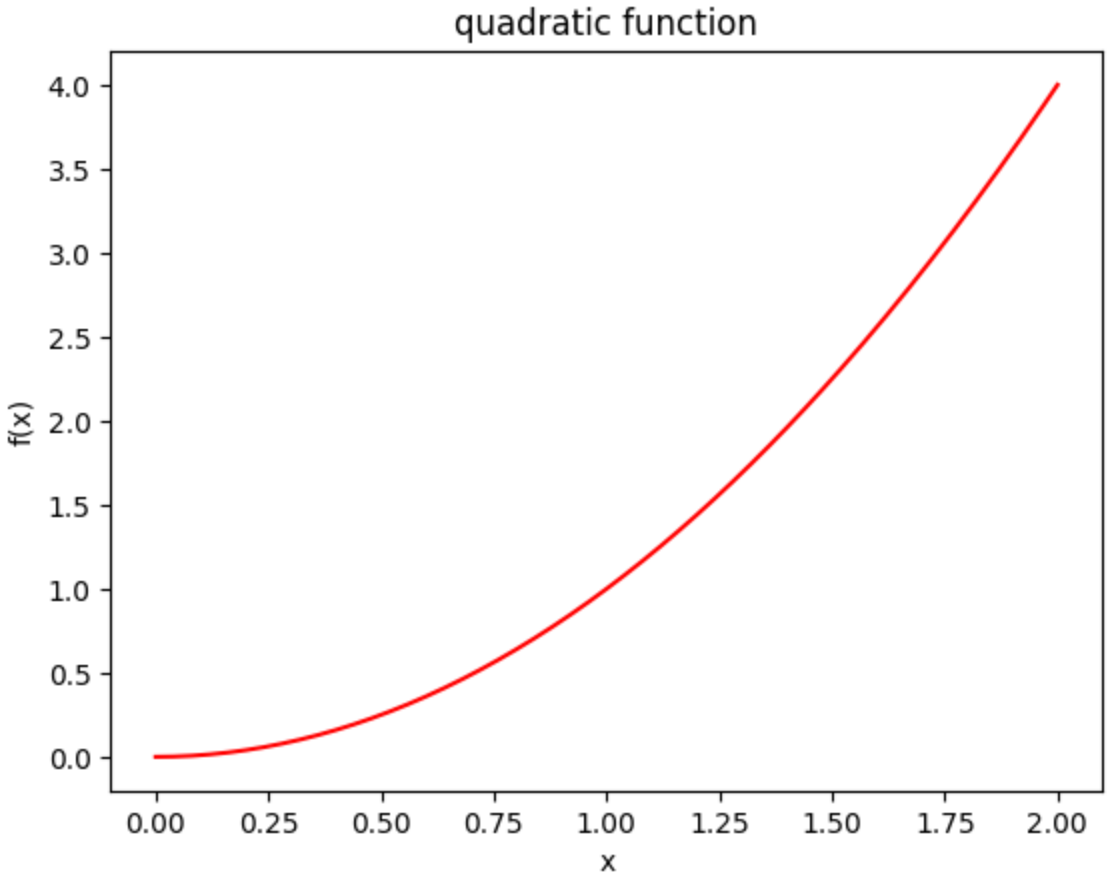


As you can observe from the figure, it is not smooth enough and unlike our expectation from the quadratic function. This is due to the fact that there is a small number of points used to plot and they are connected using straight-lines.

We can refine our x-coordinate list into many smaller points and then plot:

In [4]:
```
X = [0.01*i for i in range(0, 201)] # refined interval
Y = [x**2 for x in X]
plt.plot(X, Y, 'red')
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("quadratic function")
```

Out[4]:
```
Text(0.5, 1.0, 'quadratic function')
```



As can be seen, the above figure is more accurate.

Multiple plots on the same figure

Using the `matplotlib.pyplot` module, we can plot multiple graphs on the same figure. Typically, the figures have the same x-points list `X` and different y-values list. It is important in this case to use the `label` attribute in `.plot()` and then invoke the `.legend()` command to display these labels.

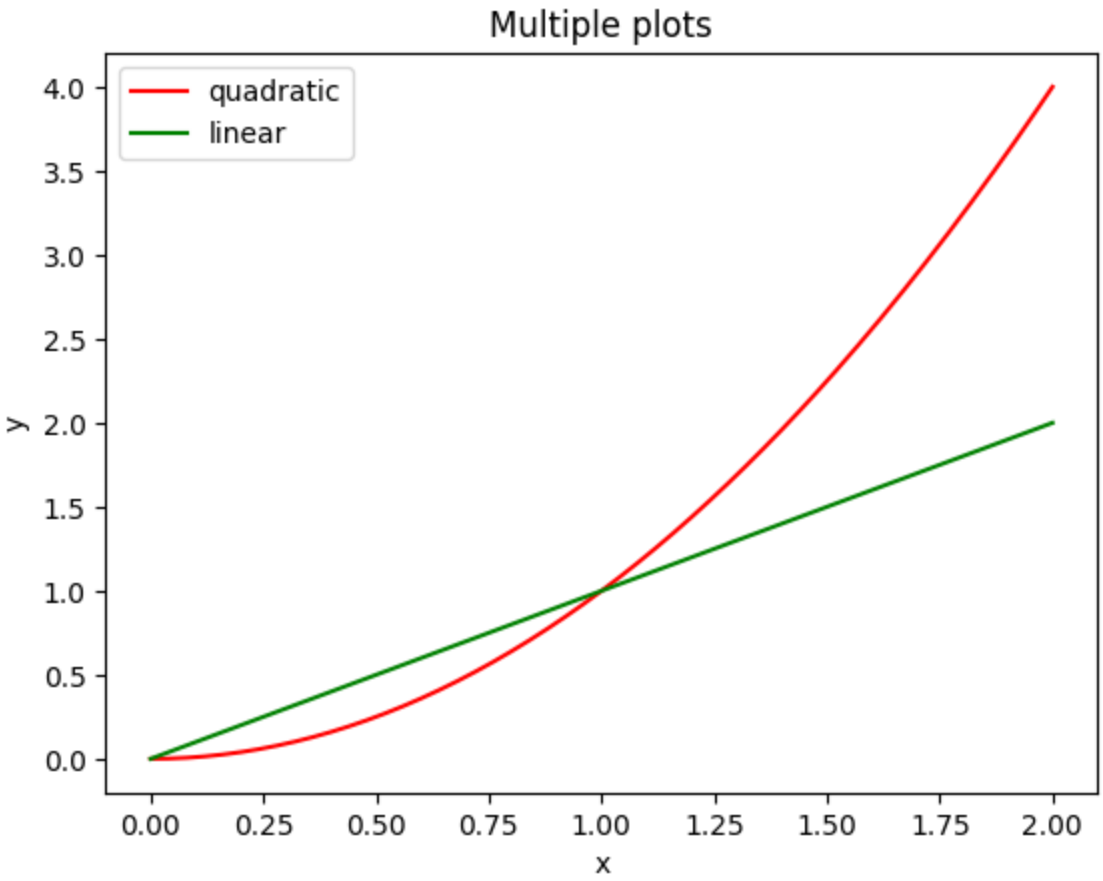
Example:

In [5]:
```
import matplotlib.pyplot as plt

X = [0.01*i for i in range(0, 201)]
Y = [x**2 for x in X]
Z = [x for x in X]

plt.plot(X, Y, color='r', label='quadratic')
plt.plot(X, Z, color='g', label='linear')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Multiple plots")
plt.legend()
```

Out[5]:
```
<matplotlib.legend.Legend at 0x2a2906fc790>
```



Subplots

Any figure can be split into rows and columns using `plt.subplot(a, b, c)`:

- `a`: is the parameter specifying the number of rows
- `b`: is the parameter specifying the number of columns
- `c`: is the parameter specifies the figure we're plotting out of the total subfigures In general, the `c` parameter moves from the left to the right, top to bottom.

It is a good idea to use the `plt.figure(x)` command before the `plt.subplot()` to specify which figure we are splitting into subplits. For example, if we need 2 figures where: the first figure consists of 1 row and 2 columns:

- `a=1`, `b=2`
- `c=1` is the left subplot, `c=2` is the right subplot

the second figure consists of 2 rows and 2 columns:

- `a=2`, `b=2`
- `c=1` is the upper-left figure, `c=2` is the top-right figure, `c=3` is bottom-left, and `c=4` is the bottom-right figure.

Example:

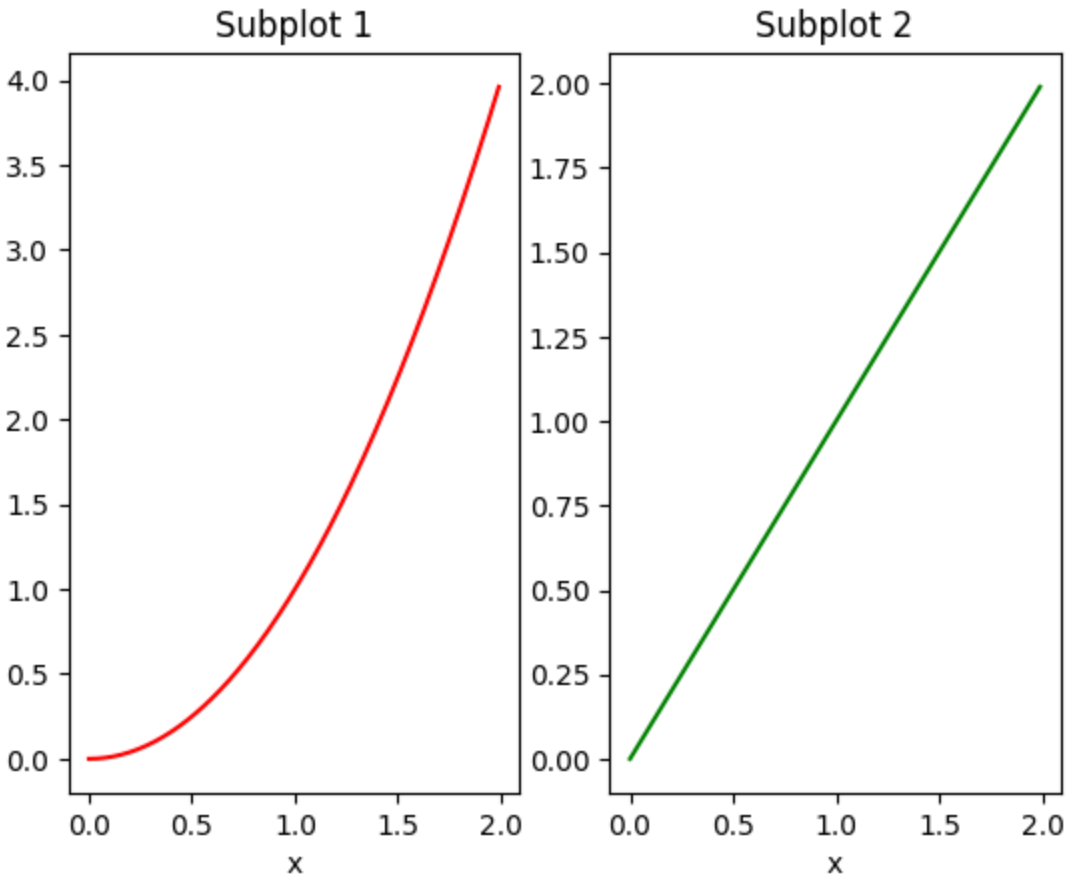
In [6]:
```
import matplotlib.pyplot as plt

X = [0.01*i for i in range(0, 200)]
Y = [x**2 for x in X]
Z = [x for x in X]

# first figure (split into two horizontally)
plt.figure(1)
plt.subplot(1, 2, 1) # left figure
plt.plot(X, Y, color='r')
plt.xlabel("x")
plt.title("Subplot 1")

plt.subplot(1, 2, 2) # right figure
plt.plot(X, Z, color='g')
plt.xlabel("x")
plt.title("Subplot 2")
```

Out[6]:
```
Text(0.5, 1.0, 'Subplot 2')
```



In [7]:
```
import matplotlib.pyplot as plt

X = [0.01*i for i in range(0, 200)]
Y = [x**2 for x in X]
Z = [x for x in X]


# second figure (split into 4)
```

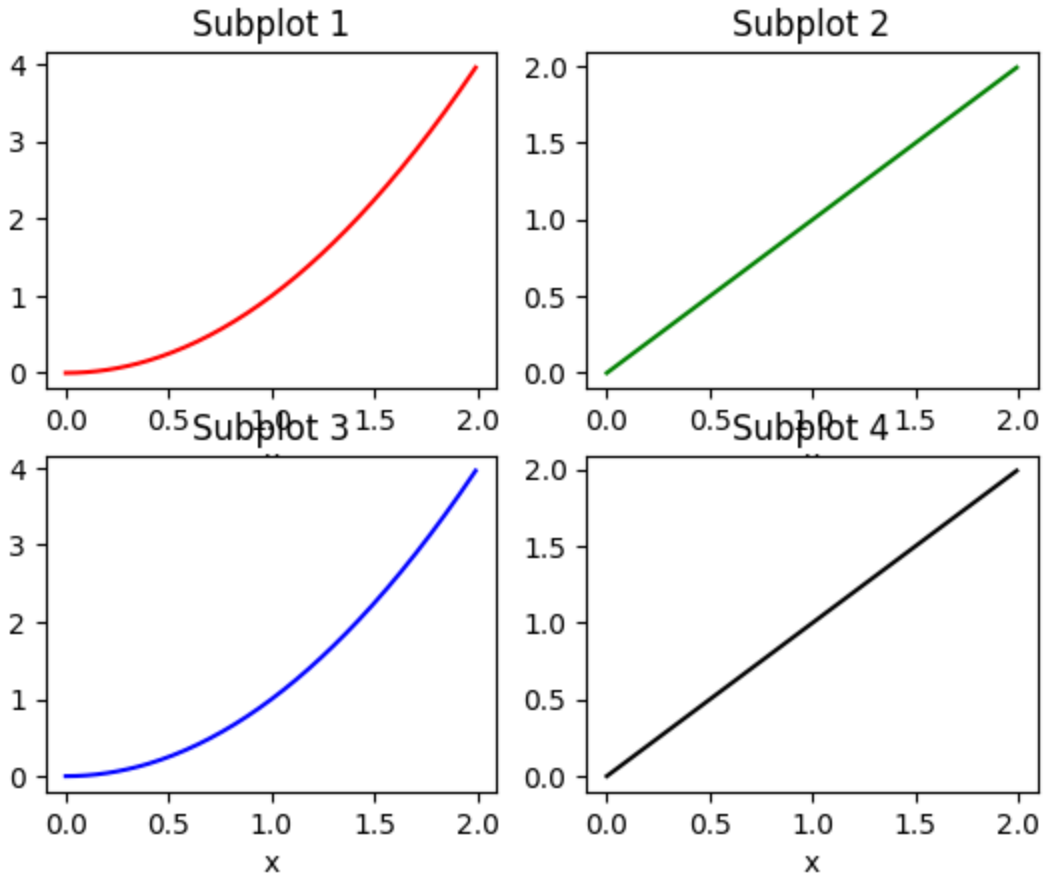
```
plt.figure(2)
plt.subplot(2, 2, 1) # top-left figure
plt.plot(X, Y, color='r')
plt.xlabel("x")
plt.title("Subplot 1")

plt.subplot(2, 2, 2) # top-right figure
plt.plot(X, Z, color='g')
plt.xlabel("x")
plt.title("Subplot 2")

plt.subplot(2, 2, 3) # top-left figure
plt.plot(X, Y, color='b')
plt.xlabel("x")
plt.title("Subplot 3")

plt.subplot(2, 2, 4) # top-right figure
plt.plot(X, Z, color='k')
plt.xlabel("x")
plt.title("Subplot 4")
```

Out[7]: Text(0.5, 1.0, 'Subplot 4')



Histograms

A histogram is a diagram consisting of rectangles whose area is proportional to the frequency of a variable and whose width is equal to the class interval. You can use the `hist`

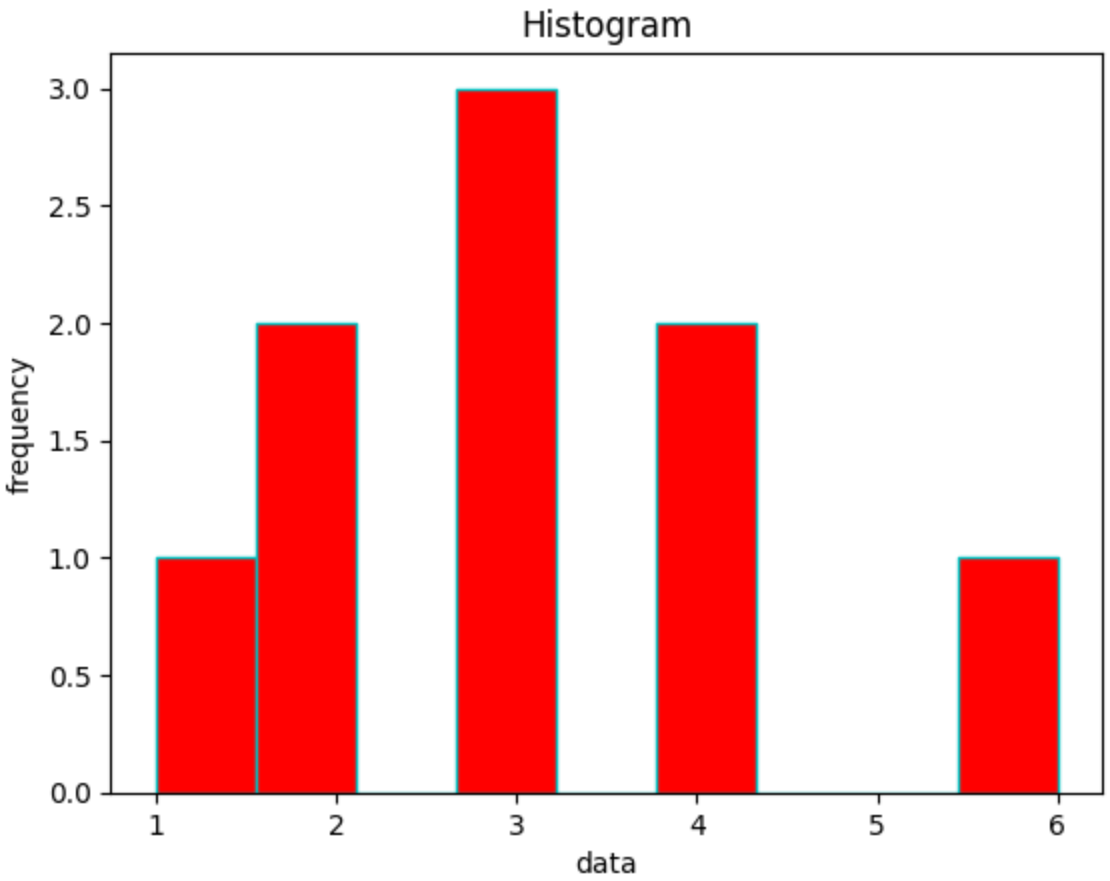
command in Python from the `matplotlib` package.

Example:

In [8]:
```
import matplotlib.pyplot as plt

data = [2, 1, 4, 3, 3, 6, 4, 2, 3]
plt.figure(3)
plt.hist(data, bins=len(data), facecolor='r', edgecolor='c')
plt.xlabel("data")
plt.ylabel("frequency")
plt.title("Histogram")
```

Out[8]:
```
Text(0.5, 1.0, 'Histogram')
```



Bar Charts

We can use the `plt.bar` command to plot a bar-chart. Here, two lists are required: the x-coordinates list and heights list.

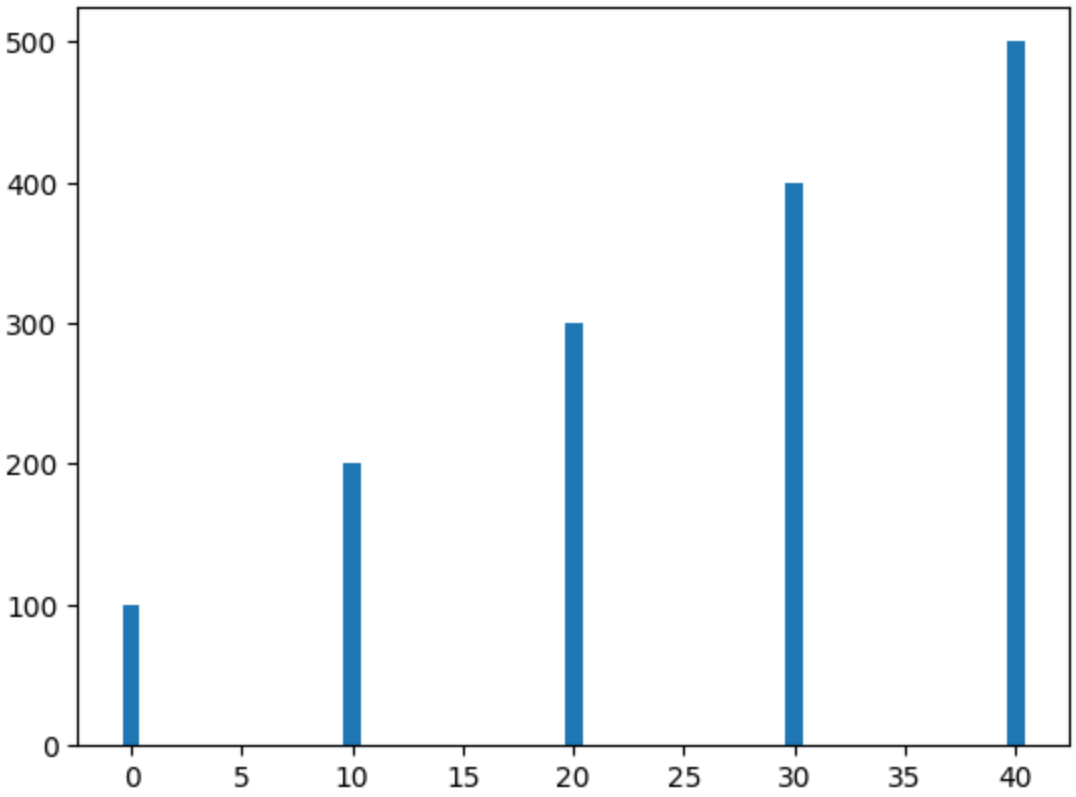
Example:

In [9]:
```
import matplotlib.pyplot as plt

left_edges = [0, 10, 20, 30, 40]
```

```
heights = [100, 200, 300, 400, 500]
plt.bar(left_edges, heights)
```

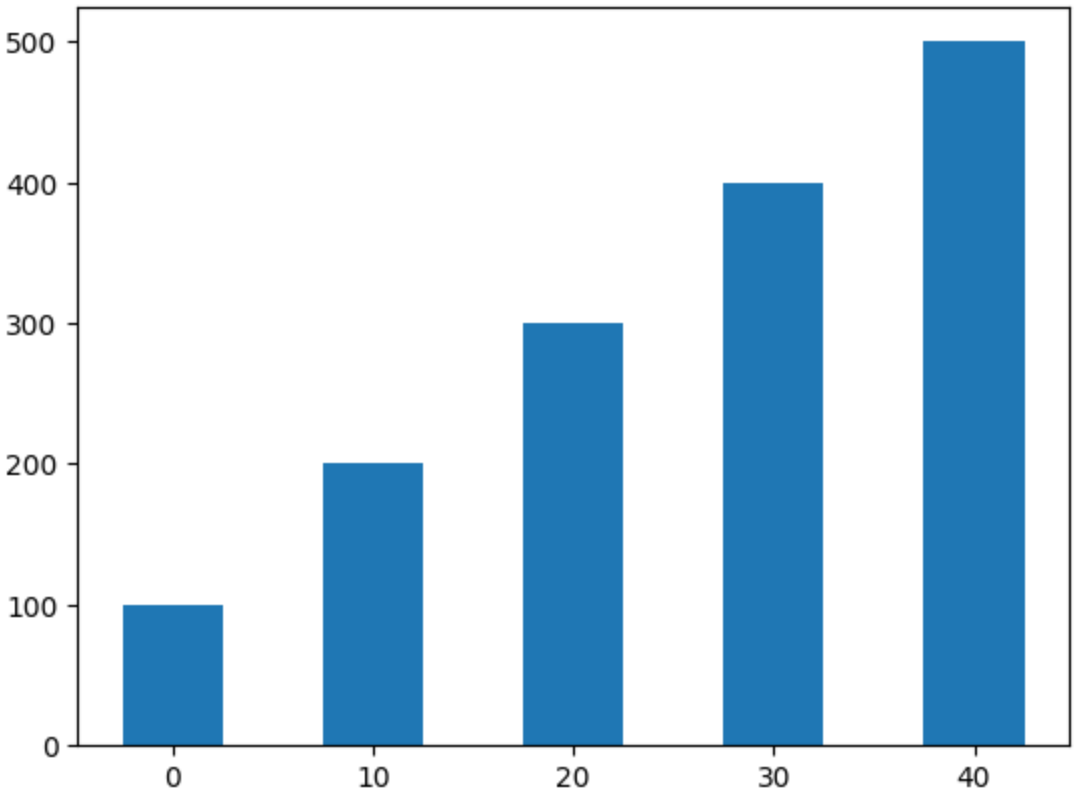
Out[9]: <BarContainer object of 5 artists>



In the example: `left_edges` are the x-coordinates at which the bars will be drawn, and `heights` contains the lengths of bars for each coordinate in `left_edges`. The default width of each bar in a bar graph is 0.8 along the x-axis, which can be changed by passing a third argument to `plt.bar`.

In [10]:
```
plt.bar(left_edges, heights, 5)
```

Out[10]: <BarContainer object of 5 artists>



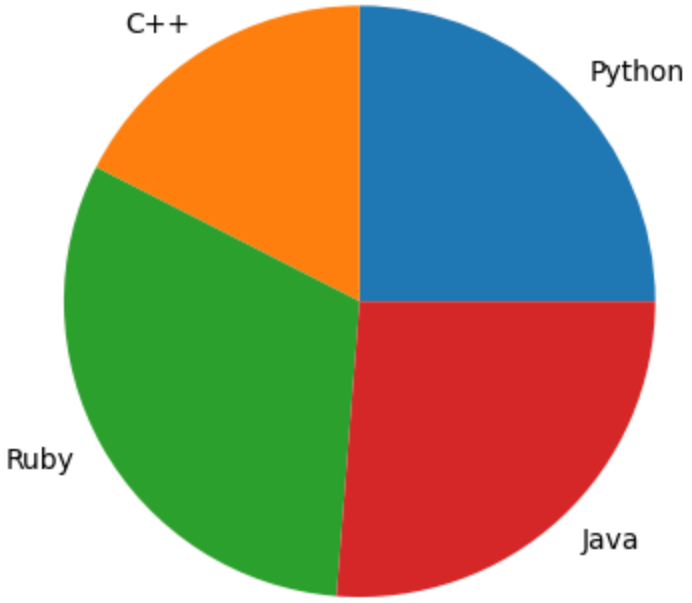
Pie Charts

We can use the `plt.pie` command to plot a pie-chart. Here, two lists are required: the `labels` list and `sizes` list.

In [11]:
```
import matplotlib.pyplot as plt

list_of_labels = ["Python", "C++", "Ruby", "Java"]
sizes = [200, 140, 250, 210]
plt.pie(sizes, labels=list_of_labels)
```

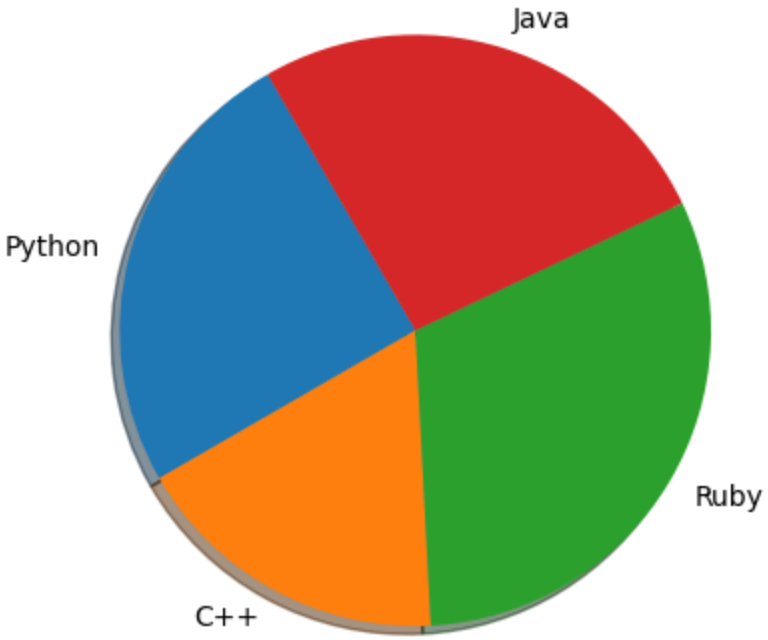
Out[11]:
```
([<matplotlib.patches.Wedge at 0x2a291a9c250>,
  <matplotlib.patches.Wedge at 0x2a291a9d870>,
  <matplotlib.patches.Wedge at 0x2a291a9fe50>,
  <matplotlib.patches.Wedge at 0x2a291acc3d0>],
 [Text(0.777817442305461, 0.7778174763049434, 'Python'),
  Text(-0.5747485603887846, 0.9379040954868573, 'C++'),
  Text(-0.9597454490375064, -0.537483649101808, 'Ruby'),
  Text(0.7466806900156542, -0.8077548806152437, 'Java')])
```



In the example, the sum of sizes list is 800. Accordingly, the label "Python" is assigned a pie slice of size 200/800, "C++" a pie slice of size 140/800, etc. There also optional **keyword parameters** for the `plt.pie` such as: `colors`, `shadow`, `startangle`.

In [12]:
```
plt.pie(sizes, labels=list_of_labels, shadow=True, startangle=120)
```

Out[12]:
```
([<matplotlib.patches.Wedge at 0x2a291aea8f0>,
  <matplotlib.patches.Wedge at 0x2a291aea830>,
  <matplotlib.patches.Wedge at 0x2a291aeb700>,
  <matplotlib.patches.Wedge at 0x2a291aebee0>],
 [Text(-1.0625185165806301, 0.2847005478099705, 'Python'),
  Text(-0.5248742059906867, -0.9666990575590971, 'C++'),
  Text(0.9453476538140453, -0.5624213842203016, 'Ruby'),
  Text(0.32619483855356896, 1.0505222164718893, 'Java')])
```



For more on the `matplotlib.pyplot` library, check the tutorial.

In [ ]: